Guia de comandos — People Analytics RH

Estrutura

```
People_Analytics_RH/
■■■ 01_Dados/
■   ■■■ 01_Originais/Funcionarios.xlsx
■   ■■■ 02_Tratados/
■   ■   ■■■ funcionarios.csv
■   ■   ■■■ eventos_rh.csv
■   ■   ■■■ historico_quadro_mensal.csv
■   ■   ■■■ historico_remuneracao.csv
■   ■   ■■■ dim_calendario.csv
■   ■   ■■■ dim_area.csv
■   ■   ■■■ dim_cargo.csv
■   ■   ■■■ dim_localidade.csv
■   ■   ■■■ dim_nivel.csv
■   ■■■ 03_Validacao/relatorio_validacao.csv
■■■ 02_Python/validar_base_people_analytics.py
■■■ 03_SQL/
■■■ 04_Modelagem/
■■■ 05_PowerBI/
■■■ 06_Documentacao/
■■■ 07_Portfolio/
```

1. Abrir a pasta no Terminal (macOS)

O projeto deste caso fica na Mesa/Desktop:

```
cd ~/Desktop/People_Analytics_RH
pwd
ls -lh 01_Dados/01_Originais/Funcionarios.xlsx
ls -lh 01_Dados/02_Tratados/*.csv
```

2. Inspecionar os CSVs sem alterá-los

Execute este bloco no Terminal (zsh), não dentro do prompt interativo >>>:

```
python3 - <<'PY'
import csv, glob, os
for arquivo in sorted(glob.glob("01_Dados/02_Tratados/*.csv")):
    with open(arquivo, encoding="utf-8-sig", newline="") as f:
        leitor = csv.reader(f)
        cabecalho = next(leitor, [])
        registros = sum(1 for _ in leitor)
    print(f"\nARQUIVO: {os.path.basename(arquivo)}")
    print(f"COLUNAS: {len(cabecalho)} | REGISTROS: {registros}")
    print("CAMPOS:", " | ".join(cabecalho))
PY
```

3. Executar a validação consolidada

Copie validar_base_people_analytics.py para 02_Python/ e execute na raiz do projeto:

```
python3 02_Python/validar_base_people_analytics.py --pasta 01_Dados/02_Tratados
```

O script lê os CSVs, executa testes, exibe os resultados e grava relatorio_validacao_gerado.csv. Ele não altera os CSVs analíticos. Código de saída 0 significa que os testes codificados passaram; código 1 indica que pelo menos um teste encontrou ocorrência.

4. Resultados observados nesta base

- funcionarios.csv: 600 registros e 24 colunas.
- eventos_rh.csv: 690 registros e 5 colunas.
- historico_remuneracao.csv: 12.466 registros e 11 colunas.
- historico_quadro_mensal.csv: 45 registros e 5 colunas.
- dim_calendario.csv: 1.369 registros e 8 colunas.
- dim_area.csv: 8 registros; dim_cargo.csv: 31; dim_localidade.csv: 10; dim_nivel.csv: 6.
- Relatório anterior: 23 verificações, todas com zero ocorrências.
- Testes manuais: sem IDs duplicados nas três tabelas testadas; sem eventos/remunerações referenciando funcionário inexistente; sem duplicidade funcionário/data, calendário/data ou quadro/mês; 510 sem data de desligamento, 90 com data; 600 eventos de admissão e 90 de desligamento.

5. Importante sobre a geração dos CSVs

A fonte Funcionarios.xlsx foi recebida e preservada como referência. A base de 600 funcionários é sintética. O histórico disponível não preserva o código-fonte nem o comando exato do gerador original; portanto, não devemos inventar um comando como se tivesse sido realmente executado.

Para que a geração seja reproduzível em projetos futuros, guarde o script real de transformação/geração em 02_Python/gerar_base_historica.py, registrando parâmetros, regras de negócio, versão e arquivos de saída. O script de validação entregue é separado: ele audita os CSVs existentes, não os gera nem os corrige.

6. Processo reutilizável

1. Registrar nome, origem e data de recebimento da fonte.
2. Preservar a fonte original sem alterações.
3. Fazer profiling de abas, colunas, tipos, nulos, duplicidades e intervalos.
4. Definir granularidade, chaves e regras de negócio.
5. Versionar o script de transformação/geração.
6. Gerar arquivos tratados em pasta separada.
7. Executar validações automatizadas e salvar relatório datado.
8. Corrigir a lógica de transformação, nunca ajustar os resultados manualmente sem registro.
9. Só carregar no Power BI depois de aprovar as regras críticas.
10. Guardar scripts, dicionário, regras, relatórios e versão do PBIX junto ao projeto.

Zero ocorrências significa que os testes documentados passaram; não é uma garantia universal de correção de toda regra de negócio.